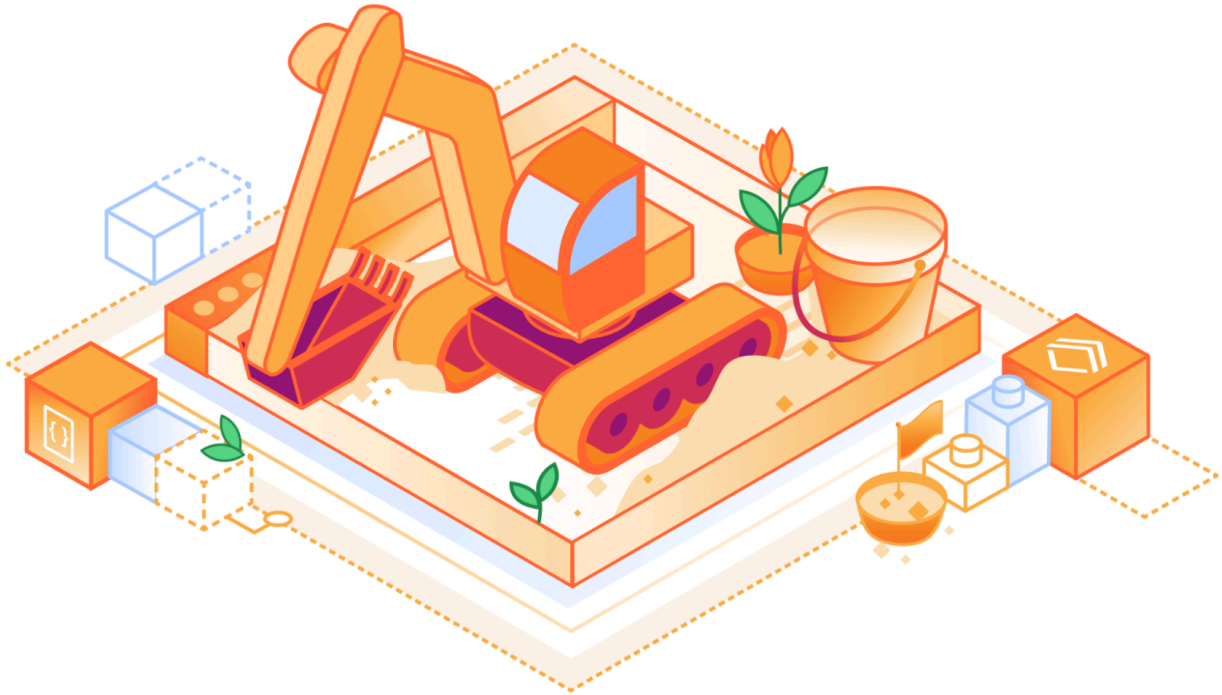
Code Mode: the better way to use MCP

2025-09-26

 Kenton Varda

 Sunil Pai

9 min read



It turns out we've all been using MCP wrong.

Most agents today use MCP by directly exposing the "tools" to the LLM.

We tried something different: Convert the MCP tools into a TypeScript API, and then ask an LLM to write code that calls that API.

The results are striking:

1. We found agents are able to handle many more tools, and more complex tools, when those tools are presented as a TypeScript API rather than

directly. Perhaps this is because LLMs have an enormous amount of real-world TypeScript in their training set, but only a small set of contrived examples of tool calls.

2. The approach really shines when an agent needs to string together multiple calls. With the traditional approach, the output of each tool call must feed into the LLM's neural network, just to be copied over to the inputs of the next call, wasting time, energy, and tokens. When the LLM can write code, it can skip all that, and only read back the final results it needs.

In short, LLMs are better at writing code to call MCP, than at calling MCP directly.

What's MCP?

For those that aren't familiar: Model Context Protocol is a standard protocol for giving AI agents access to external tools, so that they can directly perform work, rather than just chat with you.

Seen another way, MCP is a uniform way to:

- expose an API for doing something,
- along with documentation needed for an LLM to understand it,
- with authorization handled out-of-band.

MCP has been making waves throughout 2025 as it has suddenly greatly expanded the capabilities of AI agents.

The "API" exposed by an MCP server is expressed as a set of "tools". Each tool is essentially a remote procedure call (RPC) function – it is called with some parameters and returns a response. Most modern LLMs have the capability to use "tools" (sometimes called "function calling"), meaning they are trained to output text in a certain format when they want to invoke a tool. The program invoking the LLM sees this format and invokes the tool as specified, then feeds the results back into the LLM as input.

Anatomy of a tool call

Under the hood, an LLM generates a stream of "tokens" representing its output. A token might represent a word, a syllable, some sort of punctuation, or some other component of text.

A tool call, though, involves a token that does *not* have any textual equivalent. The LLM is trained (or, more often, fine-tuned) to understand a special token that it can output that means "the following should be interpreted as a tool call," and another special token that means "this is the end of the tool call." Between these two tokens, the LLM will typically write tokens corresponding to some sort of JSON message that describes the call.

For instance, imagine you have connected an agent to an MCP server that provides weather info, and you then ask the agent what the weather is like in Austin, TX. Under the hood, the LLM might generate output like the following. Note that here we've used words in <| and |> to represent our special tokens, but in fact, these tokens do not represent text at all; this is just for illustration.

I will use the Weather MCP server to find out the weather in Austin, TX.

```
I will use the Weather MCP server to find out the weather in Austin, TX.

<|tool_call|>
{
  "name": "get_current_weather",
  "arguments": {
    "location": "Austin, TX, USA"
  }
}
<|end_tool_call|>
```

Upon seeing these special tokens in the output, the LLM's harness will interpret the sequence as a tool call. After seeing the end token, the harness pauses execution of the LLM. It parses the JSON message and returns it as a separate component of the structured API result. The agent calling the LLM API sees the tool call, invokes the relevant MCP server, and then sends the results back to the LLM API. The LLM's harness will then use another set of special tokens to feed the result back into the LLM:

```
<|tool_result|>
{
  "location": "Austin, TX, USA",
  "temperature": 93,
  "unit": "fahrenheit",
  "conditions": "sunny"
}
<|end_tool_result|>
```

The LLM reads these tokens in exactly the same way it would read input from the user – except that the user cannot produce these special tokens, so the LLM knows it is the result of the tool call. The LLM then continues generating output like normal.

Different LLMs may use different formats for tool calling, but this is the basic idea.

What's wrong with this?

The special tokens used in tool calls are things LLMs have never seen in the wild. They must be specially trained to use tools, based on synthetic training data. They aren't always that good at it. If you present an LLM with too many tools, or overly complex tools, it may struggle to choose the right one or to use it correctly. As a result, MCP server designers are encouraged to present greatly simplified APIs as compared to the more traditional API they might expose to developers.

Meanwhile, LLMs are getting really good at writing code. In fact, LLMs asked to write code against the full, complex APIs normally exposed to developers don't seem to have too much trouble with it. Why, then, do MCP interfaces have to "dumb it down"? Writing code and calling tools are almost the same thing, but it seems like LLMs can do one much better than the other?

The answer is simple: LLMs have seen a lot of code. They have not seen a lot of "tool calls". In fact, the tool calls they have seen are probably limited to a contrived training set constructed by the LLM's own developers, in order to try to train it. Whereas they have seen real-world code from millions of open source projects.

Making an LLM perform tasks with tool calling is like putting Shakespeare through a month-long class in Mandarin and then asking him to write a play in it. It's just not going to be his best work.

But MCP is still useful, because it is uniform

MCP is designed for tool-calling, but it doesn't actually *have to* be used that way.

The "tools" that an MCP server exposes are really just an RPC interface with attached documentation. We don't really *have to* present them as tools. We can take the tools, and turn them into a programming language API instead.

But why would we do that, when the programming language APIs already exist independently? Almost every MCP server is just a wrapper around an existing traditional API – why not expose those APIs?

Well, it turns out MCP does something else that's really useful: **It provides a uniform way to connect to and learn about an API.**

An AI agent can use an MCP server even if the agent's developers never heard of the particular MCP server, and the MCP server's developers never heard of the particular agent. This has rarely been true of traditional APIs in the past. Usually, the client developer always knows exactly what API they are coding for. As a result, every API is able to do things like basic connectivity, authorization, and documentation a little bit differently.

This uniformity is useful even when the AI agent is writing code. We'd like the AI agent to run in a sandbox such that it can only access the tools we give it. MCP makes it possible for the agentic framework to implement this, by handling connectivity and authorization in a standard way, independent of the AI code. We also don't want the AI to have to search the Internet for documentation; MCP provides it directly in the protocol.

OK, how does it work?

We have already extended the Cloudflare Agents SDK to support this new model!

For example, say you have an app built with ai-sdk that looks like this:

```
const stream = streamText({
  model: openai("gpt-5"),
  system: "You are a helpful assistant",
  messages: [
    { role: "user", content: "Write a function that adds two numbers" }
  ],
  tools: {
    // tool definitions
  }
})
```

You can wrap the tools and prompt with the codemode helper, and use them in your app:

```
import { codemode } from "agents/codemode/ai";

const {system, tools} = codemode({
  system: "You are a helpful assistant",
  tools: {
    // tool definitions
  },
  // ...config
})

const stream = streamText({
  model: openai("gpt-5"),
  system,
  tools,
  messages: [
```

```
    { role: "user", content: "Write a function that adds two numbers" }
  ]
})
```

With this change, your app will now start generating and running code that itself will make calls to the tools you defined, MCP servers included. We will introduce variants for other libraries in the very near future. Read the docs for more details and examples.

Converting MCP to TypeScript

When you connect to an MCP server in "code mode", the Agents SDK will fetch the MCP server's schema, and then convert it into a TypeScript API, complete with doc comments based on the schema.

For example, connecting to the MCP server at https://gitmcp.io/cloudflare/agents, will generate a TypeScript definition like this:

```
interface FetchAgentsDocumentationInput {
  [k: string]: unknown;
}
interface FetchAgentsDocumentationOutput {
  [key: string]: any;
}

interface SearchAgentsDocumentationInput {
  /**
   * The search query to find relevant documentation
```

```
   */
  query: string;
}
interface SearchAgentsDocumentationOutput {
  [key: string]: any;
}

interface SearchAgentsCodeInput {
  /**
   * The search query to find relevant code files
   */
  query: string;
  /**
   * Page number to retrieve (starting from 1). Each page contains 30
   * results.
   */
  page?: number;
}
interface SearchAgentsCodeOutput {
  [key: string]: any;
}

interface FetchGenericUrlContentInput {
  /**
   * The URL of the document or page to fetch
   */
  url: string;
}
interface FetchGenericUrlContentOutput {
  [key: string]: any;
}

declare const codemode: {
  /**
   * Fetch entire documentation file from GitHub repository:
```

```
     * cloudflare/agents. Useful for general questions. Always call
     * this tool first if asked about cloudflare/agents.
     */
    fetch_agents_documentation: (
      input: FetchAgentsDocumentationInput
    ) => Promise<FetchAgentsDocumentationOutput>;

    /**
     * Semantically search within the fetched documentation from
     * GitHub repository: cloudflare/agents. Useful for specific queries.
     */
    search_agents_documentation: (
      input: SearchAgentsDocumentationInput
    ) => Promise<SearchAgentsDocumentationOutput>;

    /**
     * Search for code within the GitHub repository: "cloudflare/agents"
     * using the GitHub Search API (exact match). Returns matching files
     * for you to query further if relevant.
     */
    search_agents_code: (
      input: SearchAgentsCodeInput
    ) => Promise<SearchAgentsCodeOutput>;

    /**
     * Generic tool to fetch content from any absolute URL, respecting
     * robots.txt rules. Use this to retrieve referenced urls (absolute
     * urls) that were mentioned in previously fetched documentation.
     */
    fetch_generic_url_content: (
      input: FetchGenericUrlContentInput
    ) => Promise<FetchGenericUrlContentOutput>;
};
```

This TypeScript is then loaded into the agent's context. Currently, the entire API is loaded, but future improvements could allow an agent to search and browse the API more dynamically – much like an agentic coding assistant would.

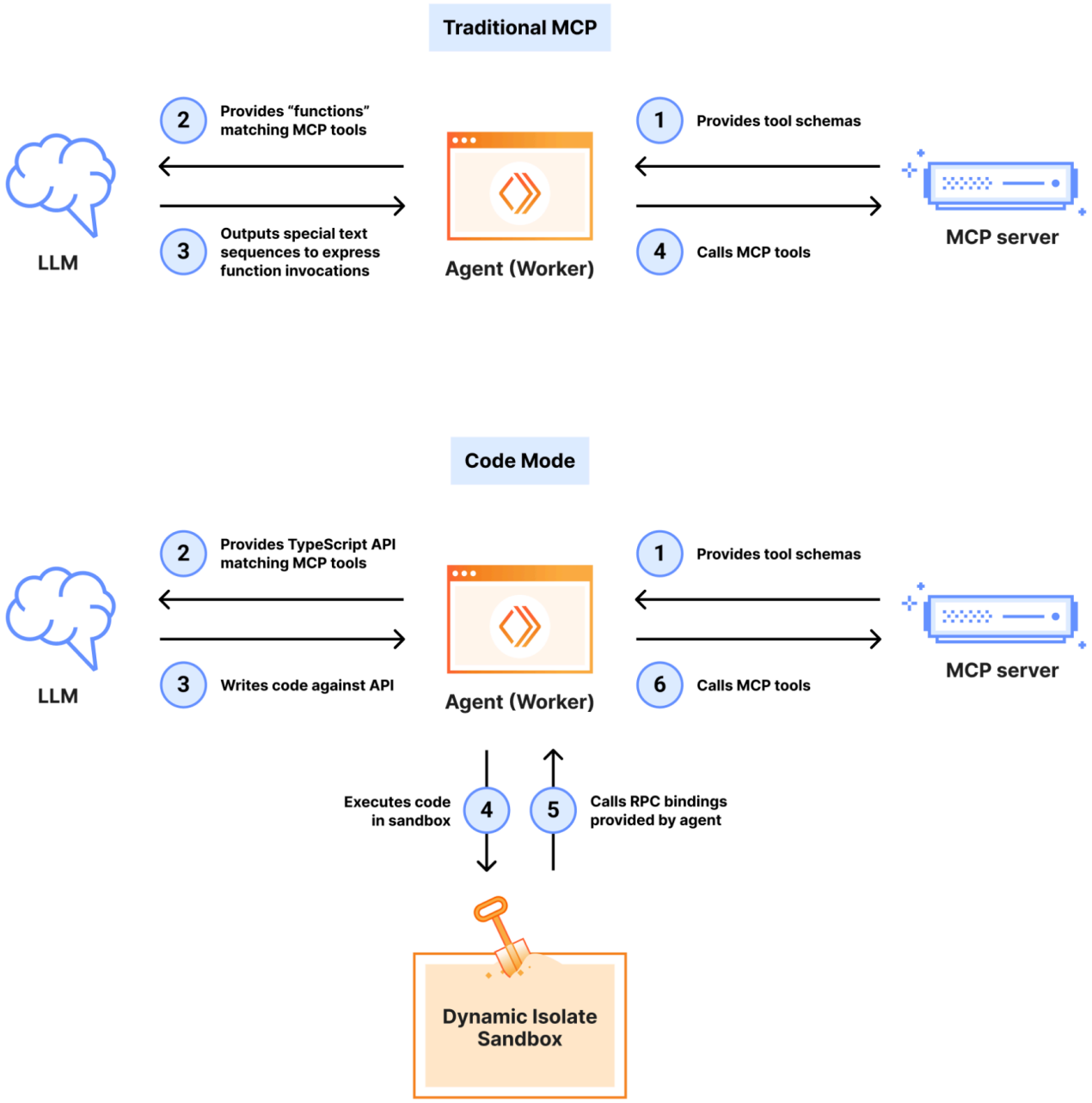
Running code in a sandbox

Instead of being presented with all the tools of all the connected MCP servers, our agent is presented with just one tool, which simply executes some TypeScript code.

The code is then executed in a secure sandbox. The sandbox is totally isolated from the Internet. Its only access to the outside world is through the TypeScript APIs representing its connected MCP servers.

These APIs are backed by RPC invocation which calls back to the agent loop. There, the Agents SDK dispatches the call to the appropriate MCP server.

The sandboxed code returns results to the agent in the obvious way: by invoking `console.log()`. When the script finishes, all the output logs are passed back to the agent.


Traditional MCP
LLM
2
Provides "functions" matching MCP tools
3
Outputs special text sequences to express function invocations
Agent (Worker)
1
Provides tool schemas
4
Calls MCP tools
MCP server
Code Mode
LLM
2
Provides TypeScript API matching MCP tools
3
Writes code against API
Agent (Worker)
1
Provides tool schemas
6
Calls MCP tools
MCP server
Executes code in sandbox
4
5
Calls RPC bindings provided by agent
Dynamic Isolate Sandbox

Dynamic Worker loading: no containers here

This new approach requires access to a secure sandbox where arbitrary code can run. So where do we find one? Do we have to run containers? Is that expensive?

No. There are no containers. We have something much better: isolates.

The Cloudflare Workers platform has always been based on V8 isolates, that is, isolated JavaScript runtimes powered by the V8 JavaScript engine.

Isolates are far more lightweight than containers. An isolate can start in a handful of milliseconds using only a few megabytes of memory.

Isolates are so fast that we can just create a new one for every piece of code the agent runs. There's no need to reuse them. There's no need to prewarm them. Just create it, on demand, run the code, and throw it away. It all happens so fast that the overhead is negligible; it's almost as if you were just eval()ing the code directly. But with security.

The Worker Loader API

Until now, though, there was no way for a Worker to directly load an isolate containing arbitrary code. All Worker code instead had to be uploaded via the Cloudflare API, which would then deploy it globally, so that it could run anywhere.

That's not what we want for Agents! We want the code to just run right where the agent is.

To that end, we've added a new API to the Workers platform: the Worker Loader API. With it, you can load Worker code on-demand. Here's what it looks like:

```
// Gets the Worker with the given ID, creating it if no such Worker exists yet.
let worker = env.LOADER.get(id, async () => {
  // If the Worker does not already exist, this callback is invoked to fetch
  // its code.

  return {
    compatibilityDate: "2025-06-01",

    // Specify the worker's code (module files).
    mainModule: "foo.js",
    modules: {
      "foo.js":
        "export default {\n" +
        "  fetch(req, env, ctx) { return new Response('Hello'); }\n" +
        "}\n",
    },

    // Specify the dynamic Worker's environment (`env`).
    env: {
      // It can contain basic serializable data types...
      SOME_NUMBER: 123,

      // ... and bindings back to the parent worker's exported RPC
      // interfaces, using the new `ctx.exports` loopback bindings API.
      SOME_RPC_BINDING: ctx.exports.MyBindingImpl({props})
    },
```

```
    // Redirect the Worker's `fetch()` and `connect()` to proxy through
    // the parent worker, to monitor or filter all Internet access. You
    // can also block Internet access completely by passing `null`.
    globalOutbound: ctx.exports.OutboundProxy({props}),
  };
});

// Now you can get the Worker's entrypoint and send requests to it.
let defaultEntrypoint = worker.getEntrypoint();
await defaultEntrypoint.fetch("http://example.com");

// You can get non-default entrypoints as well, and specify the
// `ctx.props` value to be delivered to the entrypoint.
let someEntrypoint = worker.getEntrypoint("SomeEntrypointClass", {
  props: {someProp: 123}
});
```

You can start playing with this API right now when running `workerd` locally with Wrangler (check out the docs), and you can sign up for beta access to use it in production.

Workers are better sandboxes

The design of Workers makes it unusually good at sandboxing, especially for this use case, for a few reasons:

Faster, cheaper, disposable sandboxes

The Workers platform uses isolates instead of containers. Isolates are much lighter-weight and faster to start up. It takes mere milliseconds to start a fresh isolate, and it's so cheap we can just create a new one for every single code snippet the agent generates. There's no need to worry about pooling isolates for reuse, prewarming, etc.

We have not yet finalized pricing for the Worker Loader API, but because it is based on isolates, we will be able to offer it at a significantly lower cost than container-based solutions.

Isolated by default, but connected with bindings

Workers are just better at handling isolation.

In Code Mode, we prohibit the sandboxed worker from talking to the Internet. The global `fetch()` and `connect()` functions throw errors.

But on most platforms, this would be a problem. On most platforms, the way you get access to private resources is, you *start* with general network access. Then, using that network access, you send requests to specific services, passing them some sort of API key to authorize private access.

But Workers has always had a better answer. In Workers, the "environment" (`env` object) doesn't just contain strings, it contains live objects, also known as "bindings". These objects can provide direct access to private resources without involving generic network requests.

In Code Mode, we give the sandbox access to bindings representing the MCP servers it is connected to. Thus, the agent can specifically access those MCP servers *without* having network access in general.

Limiting access via bindings is much cleaner than doing it via, say, network-level filtering or HTTP proxies. Filtering is hard on both the LLM and the supervisor, because the boundaries are often unclear: the supervisor may have a hard time identifying exactly what traffic is legitimately necessary to talk to an API. Meanwhile, the LLM may have difficulty guessing what kinds of requests will be blocked. With the bindings approach, it's well-defined: the binding provides a JavaScript interface, and that interface is allowed to be used. It's just better this way.

No API keys to leak

An additional benefit of bindings is that they hide API keys. The binding itself provides an already-authorized client interface to the MCP server. All calls made on it go to the agent supervisor first, which holds the access tokens and adds them into requests sent on to MCP.

This means that the AI cannot possibly write code that leaks any keys, solving a common security problem seen in AI-authored code today.

Try it now!

Sign up for the production beta

The Dynamic Worker Loader API is in closed beta. To use it in production, sign up today.

Or try it locally

If you just want to play around, though, Dynamic Worker Loading is fully available today when developing locally with Wrangler and `workerd` – check out the docs for Dynamic Worker Loading and code mode in the Agents SDK to get started.

Cloudflare's connectivity cloud protects entire corporate networks, helps customers build Internet-scale applications efficiently, accelerates any website or Internet application, wards off DDoS attacks, keeps hackers at bay, and can help you on your journey to Zero Trust.

Visit 1.1.1.1 from any device to get started with our free app that makes your Internet faster and safer.

To learn more about our mission to help build a better Internet, start here. If you're looking for a new career direction, check out our open positions.

Discuss on Hacker News

On Air

Workers Launchpad Demo Day - Cohort #1

Tune In



AI Birthday Week Cloudflare Workers Agents MCP

Follow on X

Kenton Varda | @kentonvarda
Sunil Pai | @threepointone
Cloudflare | @cloudflare

RELATED POSTS

November 12, 2025 7:30 PM

Connecting to production: the architecture of remote bindings

Remote bindings allow you to connect your local Worker code to deployed Cloudflare resources like R2 and D1. Come along on the technical journey of how we built this feature to create a seamless local development experience....

By Samuel Macleod, Dario Piotrowicz

Cloudflare Workers, R2, D1

November 10, 2025 7:30 PM

A closer look at Python Workflows, now in beta

Cloudflare Workflows, our durable execution engine for running multi-step applications, now supports Python. That means less friction, more possibilities, and another reason to build on Cloudflare....

By Caio Nogueira, Mia Malden

Workflows, Cloudflare Workers, Python

November 05, 2025 7:30 PM

How Workers VPC Services connects to your regional private networks from anywhere in the world

Workers VPC Services enter open beta today. We look under the hood to see how Workers VPC connects your globally-deployed Workers to your regional private networks by using Cloudflare's global network, while abstracting cross-cloud networking complexity....

By Thomas Gauvin, Matt Alonso, Eric Falcão

Cloudflare Workers, Workers VPC, Cloudflare Tunnel, Network, Hybrid Cloud, Security, VPC, Private Network

November 04, 2025 7:30 PM

Building a better testing experience for Workflows, our durable execution engine for multi-step applications

End-to-end testing for Cloudflare Workflows was challenging. We're introducing first-class support for Workflows in cloudflare:test, enabling full introspection, mocking, and isolated, reliable tests for your most complex applications....

By Olga Silva, Mia Malden

Developers, Internship Experience, Product News, Developer Platform, Cloudflare Workers, Workflows

© 2025 Cloudflare, Inc. | Privacy Policy | Terms of Use | Report Security Issues | Cookie Preferences | Trademark